

YTBLND

1.0

Generated by Doxygen 1.14.0

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 CsvParser Class Reference	7
4.1.1 Detailed Description	8
4.1.2 Constructor & Destructor Documentation	8
4.1.2.1 CsvParser() [1/2]	8
4.1.2.2 CsvParser() [2/2]	8
4.1.3 Member Function Documentation	8
4.1.3.1 getParsed()	8
4.1.3.2 parse()	9
4.1.3.3 setData()	9
4.1.4 Member Data Documentation	9
4.1.4.1 parse_id	9
4.1.4.2 raw_data	9
4.2 CsvSource Class Reference	10
4.2.1 Detailed Description	11
4.2.2 Constructor & Destructor Documentation	11
4.2.2.1 CsvSource()	11
4.2.3 Member Function Documentation	11
4.2.3.1 getSourceId()	11
4.2.3.2 read()	11
4.2.3.3 setSource()	12
4.2.4 Member Data Documentation	12
4.2.4.1 source_id	12
4.2.4.2 src	12
4.3 DataExtractor Class Reference	12
4.3.1 Detailed Description	12
4.3.2 Constructor & Destructor Documentation	12
4.3.2.1 DataExtractor()	12
4.3.2.2 ~DataExtractor()	13
4.3.3 Member Function Documentation	13
4.3.3.1 extract()	13
4.3.3.2 setParser()	13
4.3.3.3 setSource()	14
4.3.4 Member Data Documentation	14

4.3.4.1 par_format	14
4.3.4.2 src_format	14
4.4 DataSaver Class Reference	14
4.4.1 Detailed Description	15
4.4.2 Constructor & Destructor Documentation	15
4.4.2.1 DataSaver()	15
4.4.2.2 ~DataSaver()	15
4.4.3 Member Function Documentation	15
4.4.3.1 save()	15
4.4.3.2 setFileBin()	16
4.4.3.3 setFileFormatter()	16
4.4.4 Member Data Documentation	16
4.4.4.1 bins	16
4.4.4.2 formatters	16
4.5 File_ID Class Reference	17
4.5.1 Detailed Description	17
4.5.2 Member Data Documentation	17
4.5.2.1 CSV	17
4.5.2.2 HTML	17
4.5.2.3 JSON	17
4.6 FileBin Class Reference	17
4.6.1 Detailed Description	18
4.6.2 Constructor & Destructor Documentation	18
4.6.2.1 ~FileBin()	18
4.6.3 Member Function Documentation	18
4.6.3.1 setFileName()	18
4.6.3.2 setOutputFile()	18
4.6.3.3 setRawData()	18
4.6.3.4 write()	19
4.7 FileSource Class Reference	19
4.7.1 Detailed Description	20
4.7.2 Member Function Documentation	20
4.7.2.1 getSourceId()	20
4.7.2.2 read()	20
4.7.2.3 setSource()	20
4.8 HtmlParser Class Reference	21
4.8.1 Detailed Description	22
4.8.2 Constructor & Destructor Documentation	22
4.8.2.1 HtmlParser() [1/2]	22
4.8.2.2 HtmlParser() [2/2]	22
4.8.3 Member Function Documentation	22
4.8.3.1 extract_links()	22

4.8.3.2 <code>get_text()</code>	22
4.8.3.3 <code>getParseId()</code>	23
4.8.3.4 <code>parse()</code>	23
4.8.3.5 <code>setData()</code>	23
4.8.4 Member Data Documentation	23
4.8.4.1 <code>parse_id</code>	23
4.8.4.2 <code>raw_data</code>	24
4.9 HtmlSource Class Reference	24
4.9.1 Detailed Description	25
4.9.2 Constructor & Destructor Documentation	25
4.9.2.1 <code>HtmlSource()</code>	25
4.9.3 Member Function Documentation	25
4.9.3.1 <code>getSourceId()</code>	25
4.9.3.2 <code>read()</code>	25
4.9.3.3 <code>setSource()</code>	26
4.9.4 Member Data Documentation	26
4.9.4.1 <code>source_id</code>	26
4.9.4.2 <code>src_path</code>	26
4.10 Parser Class Reference	26
4.10.1 Detailed Description	27
4.10.2 Member Function Documentation	27
4.10.2.1 <code>getParseId()</code>	27
4.10.2.2 <code>parse()</code>	27
4.10.2.3 <code>setData()</code>	27
4.11 WatchLaterParser Class Reference	28
4.11.1 Detailed Description	28
4.11.2 Constructor & Destructor Documentation	28
4.11.2.1 <code>WatchLaterParser()</code>	28
4.11.3 Member Function Documentation	29
4.11.3.1 <code>parse()</code>	29
4.11.4 Member Data Documentation	29
4.11.4.1 <code>filePath</code>	29
5 File Documentation	31
5.1 CsvParser.cpp File Reference	31
5.1.1 Detailed Description	32
5.2 CsvParser.cpp	32
5.3 CsvSource.cpp File Reference	33
5.3.1 Detailed Description	34
5.4 CsvSource.cpp	34
5.5 DataExtractor.cpp File Reference	35
5.5.1 Detailed Description	35

5.6 DataExtractor.cpp	36
5.7 DataExtractor.hpp File Reference	36
5.7.1 Detailed Description	37
5.8 DataExtractor.hpp	38
5.9 DataSaver.hpp File Reference	38
5.9.1 Detailed Description	39
5.10 DataSaver.hpp	39
5.11 File_ID.hpp File Reference	39
5.11.1 Detailed Description	40
5.12 File_ID.hpp	40
5.13 FileBin.hpp File Reference	40
5.13.1 Detailed Description	41
5.14 FileBin.hpp	41
5.15 FileFormatter.hpp File Reference	42
5.15.1 Detailed Description	43
5.15.2 Function Documentation	43
5.15.2.1 format()	43
5.15.2.2 setData()	43
5.15.2.3 ~FileFormatter()	43
5.16 FileFormatter.hpp	44
5.17 FileSource.hpp File Reference	44
5.17.1 Detailed Description	45
5.18 FileSource.hpp	45
5.19 HtmlParser.cpp File Reference	46
5.19.1 Detailed Description	46
5.20 HtmlParser.cpp	46
5.21 HtmlSource.cpp File Reference	48
5.21.1 Detailed Description	48
5.22 HtmlSource.cpp	48
5.23 Parser.hpp File Reference	49
5.23.1 Detailed Description	50
5.24 Parser.hpp	50
5.25 WatchLaterParser.hpp File Reference	51
5.25.1 Detailed Description	51
5.26 WatchLaterParser.hpp	52

Index	53
--------------	-----------

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

DataExtractor	12
DataSaver	14
File_ID	17
FileBin	17
FileSource	19
CsvSource	10
HtmlSource	24
Parser	26
CsvParser	7
HtmlParser	21
WatchLaterParser	28

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

CsvParser	CSV file parser implementation	7
CsvSource	CSV file source/reader implementation	10
DataExtractor	Coordinates FileSource and Parser to extract and parse file data	12
DataSaver	Coordinates FileFormatter and FileBin to save structured data to files	14
File_ID	File format ID constants for validating source-parser compatibility	17
FileBin	Abstract interface for writing data to files	17
FileSource	Abstract interface for reading file source data	19
HtmlParser	HTML file parser implementation using Gumbo	21
HtmlSource	HTML file source/reader implementation	24
Parser	Abstract interface for parsing file data into structured format	26
WatchLaterParser	Specialized parser for YouTube Takeout Watch Later CSV files	28

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

CsvParser.cpp	
CSV file parser implementation	31
CsvSource.cpp	
CSV file source/reader implementation	33
DataExtractor.cpp	
DataExtractor class implementation	35
DataExtractor.hpp	
Extracts raw file data and parses it into structured format	36
DataSaver.hpp	
Saves structured data to files in desired format	38
File_ID.hpp	
File format ID constants	39
FileBin.hpp	
Abstract base class for writing data to files	40
FileFormatter.hpp	
Abstract base class for formatting structured data to file format	42
FileSource.hpp	
Abstract base class for reading file source data	44
HtmlParser.cpp	
HTML file parser implementation using Gumbo	46
HtmlSource.cpp	
HTML file source/reader implementation	48
Parser.hpp	
Abstract base class for parsing file data into structured format	49
WatchLaterParser.hpp	
Specialized parser for YouTube Takeout Watch Later CSV files	51

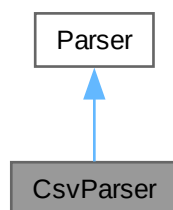
Chapter 4

Class Documentation

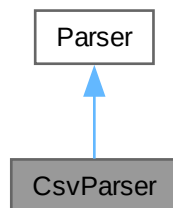
4.1 CsvParser Class Reference

CSV file parser implementation.

Inheritance diagram for CsvParser:



Collaboration diagram for CsvParser:



Public Member Functions

- [CsvParser](#) (list< string > raw_data)
- list< unordered_map< string, string > > [parse](#) () override
- void [setData](#) (list< string > data) override
- int [getParseId](#) () override

Public Member Functions inherited from [Parser](#)

- virtual void [setData](#) (std::list< std::string > data)=0

Private Attributes

- list< string > [raw_data](#)
- int [parse_id](#) = File_ID::CSV

4.1.1 Detailed Description

CSV file parser implementation.

Definition at line 15 of file [CsvParser.cpp](#).

4.1.2 Constructor & Destructor Documentation

4.1.2.1 [CsvParser\(\)](#) [1/2]

```
CsvParser::CsvParser () [inline]
```

Definition at line 17 of file [CsvParser.cpp](#).

4.1.2.2 [CsvParser\(\)](#) [2/2]

```
CsvParser::CsvParser (  
    list< string > raw_data) [inline]
```

Definition at line 18 of file [CsvParser.cpp](#).

4.1.3 Member Function Documentation

4.1.3.1 [getParseId\(\)](#)

```
int CsvParser::getParseId () [inline], [override], [virtual]
```

Gets the file format ID associated with this parser.

Returns

Integer ID representing the file format (e.g., File_ID::CSV)

Implements [Parser](#).

Definition at line 85 of file [CsvParser.cpp](#).

4.1.3.2 parse()

```
list< unordered_map< string, string > > CsvParser::parse () [inline], [override], [virtual]
```

Parses raw file data into a structured list of dictionaries.

Converts raw file data configured via setData() into a list of unordered string-to-string dictionaries. Each dictionary represents a row with column titles as keys and row contents as values.

Returns

List of unordered maps representing parsed file rows

Implements [Parser](#).

Definition at line 23 of file [CsvParser.cpp](#).

4.1.3.3 setData()

```
void CsvParser::setData (
    list< string > data) [inline], [override]
```

Definition at line 82 of file [CsvParser.cpp](#).

4.1.4 Member Data Documentation

4.1.4.1 parse_id

```
int CsvParser::parse_id = File_ID::CSV [private]
```

Definition at line 89 of file [CsvParser.cpp](#).

4.1.4.2 raw_data

```
list<string> CsvParser::raw_data [private]
```

Definition at line 88 of file [CsvParser.cpp](#).

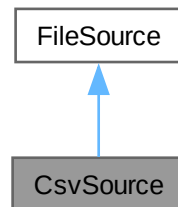
The documentation for this class was generated from the following file:

- [CsvParser.cpp](#)

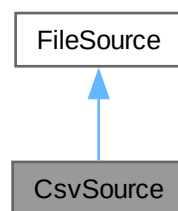
4.2 CsvSource Class Reference

CSV file source/reader implementation.

Inheritance diagram for CsvSource:



Collaboration diagram for CsvSource:



Public Member Functions

- `CsvSource` (string src)
- list< string > `read` () override
- void `setSource` (string src) override
- int `getSourceId` () override

Public Member Functions inherited from `FileSource`

- virtual void `setSource` (std::string src)=0

Private Attributes

- string `src` = ""
- int `source_id` = File_ID::CSV

4.2.1 Detailed Description

CSV file source/reader implementation.

Definition at line 15 of file [CsvSource.cpp](#).

4.2.2 Constructor & Destructor Documentation

4.2.2.1 CsvSource()

```
CsvSource::CsvSource (  
    string src) [inline]
```

Definition at line 17 of file [CsvSource.cpp](#).

4.2.3 Member Function Documentation

4.2.3.1 getSourceId()

```
int CsvSource::getSourceId () [inline], [override], [virtual]
```

Gets the file format ID associated with this source.

Returns

Integer ID representing the file format (e.g., File_ID::CSV)

Implements [FileSource](#).

Definition at line 54 of file [CsvSource.cpp](#).

4.2.3.2 read()

```
list< string > CsvSource::read () [inline], [override], [virtual]
```

Reads the configured file source path and returns raw file data.

Returns

List of strings containing raw file data (line-by-line or complete content)

Implements [FileSource](#).

Definition at line 23 of file [CsvSource.cpp](#).

4.2.3.3 setSource()

```
void CsvSource::setSource (
    string src) [inline], [override]
```

Definition at line 53 of file [CsvSource.cpp](#).

4.2.4 Member Data Documentation

4.2.4.1 source_id

```
int CsvSource::source_id = File_ID::CSV [private]
```

Definition at line 58 of file [CsvSource.cpp](#).

4.2.4.2 src

```
string CsvSource::src = "" [private]
```

Definition at line 57 of file [CsvSource.cpp](#).

The documentation for this class was generated from the following file:

- [CsvSource.cpp](#)

4.3 DataExtractor Class Reference

Coordinates [FileSource](#) and [Parser](#) to extract and parse file data.

```
#include <DataExtractor.hpp>
```

Public Member Functions

- [DataExtractor](#) (std::unique_ptr< [FileSource](#) > src, std::unique_ptr< [Parser](#) > p)
- [~DataExtractor](#) ()
- std::list< std::unordered_map< std::string, std::string > > [extract](#) ()
- void [setSource](#) (std::unique_ptr< [FileSource](#) > src)
- void [setParser](#) (std::unique_ptr< [Parser](#) > p)

Private Attributes

- std::unique_ptr< [FileSource](#) > [src_format](#)
- std::unique_ptr< [Parser](#) > [par_format](#)

4.3.1 Detailed Description

Coordinates [FileSource](#) and [Parser](#) to extract and parse file data.

Definition at line 25 of file [DataExtractor.hpp](#).

4.3.2 Constructor & Destructor Documentation

4.3.2.1 DataExtractor()

```
DataExtractor::DataExtractor (
    std::unique_ptr< FileSource > src,
    std::unique_ptr< Parser > p)
```

Constructs a [DataExtractor](#) with [FileSource](#) and [Parser](#).

Parameters

<i>src</i>	Unique pointer to FileSource implementation (must be configured)
<i>p</i>	Unique pointer to Parser implementation (must be configured)

Note

[FileSource](#) and [Parser](#) must have matching format IDs

Definition at line 14 of file [DataExtractor.cpp](#).

4.3.2.2 ~DataExtractor()

```
DataExtractor::~DataExtractor ()
```

Destructor. Cleans up owned [FileSource](#) and [Parser](#) objects.

Definition at line 20 of file [DataExtractor.cpp](#).

4.3.3 Member Function Documentation

4.3.3.1 extract()

```
list< unordered_map< string, string > > DataExtractor::extract ()
```

Extracts and parses file data using configured [FileSource](#) and [Parser](#).

Reads raw data from [FileSource](#), validates format IDs match, feeds data to [Parser](#), and returns structured format.

Returns

List of dictionaries where each dict represents a data row

Exceptions

<i>Calls</i>	exit(1) if FileSource and Parser format IDs don't match
<i>Calls</i>	exit(1) if pointers are null or validation fails

Definition at line 22 of file [DataExtractor.cpp](#).

4.3.3.2 setParser()

```
void DataExtractor::setParser (
    std::unique_ptr< Parser > p)
```

Replaces the current [Parser](#) with a new one.

Parameters

<i>p</i>	Unique pointer to new Parser implementation
----------	---

Definition at line 60 of file [DataExtractor.cpp](#).

4.3.3.3 setSource()

```
void DataExtractor::setSource (
    std::unique_ptr< FileSource > src)
```

Replaces the current [FileSource](#) with a new one.

Parameters

<i>src</i>	Unique pointer to new FileSource implementation
------------	---

Definition at line 64 of file [DataExtractor.cpp](#).

4.3.4 Member Data Documentation

4.3.4.1 par_format

```
std::unique_ptr<Parser> DataExtractor::par_format [private]
```

Definition at line 60 of file [DataExtractor.hpp](#).

4.3.4.2 src_format

```
std::unique_ptr<FileSource> DataExtractor::src_format [private]
```

Definition at line 59 of file [DataExtractor.hpp](#).

The documentation for this class was generated from the following files:

- [DataExtractor.hpp](#)
- [DataExtractor.cpp](#)

4.4 DataSaver Class Reference

Coordinates FileFormatter and [FileBin](#) to save structured data to files.

```
#include <DataSaver.hpp>
```

Public Member Functions

- [DataSaver](#) ([FileBin](#) b, [FileFormatter](#) ff)
- [~DataSaver](#) ()
- [int save](#) (std::list< std::unordered_map< std::string, std::string > > data)
- [void setFileBin](#) ([FileBin](#) b)
- [void setFileFormatter](#) ([FileFormatter](#) ff)

Private Attributes

- std::vector< std::unique_ptr< [FileBin](#) > > [bins](#)
- std::vector< std::unique_ptr< [FileFormatter](#) > > [formatters](#)

4.4.1 Detailed Description

Coordinates [FileFormatter](#) and [FileBin](#) to save structured data to files.

Definition at line 19 of file [DataSaver.hpp](#).

4.4.2 Constructor & Destructor Documentation

4.4.2.1 DataSaver()

```
DataSaver::DataSaver (
    FileBin b,
    FileFormatter ff)
```

Constructs a [DataSaver](#) with initial [FileBin](#) and [FileFormatter](#).

Parameters

<i>b</i>	The FileBin implementation for file output
<i>ff</i>	The FileFormatter implementation for data formatting

4.4.2.2 ~DataSaver()

```
DataSaver::~~DataSaver ()
```

Destructor. Cleans up owned [FileBin](#) and [FileFormatter](#) objects.

4.4.3 Member Function Documentation

4.4.3.1 save()

```
int DataSaver::save (
    std::list< std::unordered_map< std::string, std::string > > data)
```

Formats and writes data to file using configured formatters and bins.

Parameters

<i>data</i>	List of dictionaries representing data rows to save
-------------	---

Returns

Status code (0 on success, non-zero on failure)

4.4.3.2 setFileBin()

```
void DataSaver::setFileBin (
    FileBin b)
```

Adds or updates the [FileBin](#) used for file output.

Parameters

<i>b</i>	FileBin implementation for file operations
----------	--

4.4.3.3 setFileFormatter()

```
void DataSaver::setFileFormatter (
    FileFormatter ff)
```

Adds or updates the FileFormatter used for data formatting.

Parameters

<i>ff</i>	FileFormatter implementation for format conversion
-----------	--

4.4.4 Member Data Documentation**4.4.4.1 bins**

```
std::vector<std::unique_ptr<FileBin> > DataSaver::bins [private]
```

Definition at line 48 of file [DataSaver.hpp](#).

4.4.4.2 formatters

```
std::vector<std::unique_ptr<FileFormatter> > DataSaver::formatters [private]
```

Definition at line 50 of file [DataSaver.hpp](#).

The documentation for this class was generated from the following file:

- [DataSaver.hpp](#)

4.5 File_ID Class Reference

File format ID constants for validating source-parser compatibility.

```
#include <File_ID.hpp>
```

Static Public Attributes

- static int const [CSV](#) = 1
- static int const [HTML](#) = 2
- static int const [JSON](#) = 3

4.5.1 Detailed Description

File format ID constants for validating source-parser compatibility.

Definition at line [12](#) of file [File_ID.hpp](#).

4.5.2 Member Data Documentation

4.5.2.1 CSV

```
int const File_ID::CSV = 1 [static]
```

Definition at line [14](#) of file [File_ID.hpp](#).

4.5.2.2 HTML

```
int const File_ID::HTML = 2 [static]
```

Definition at line [15](#) of file [File_ID.hpp](#).

4.5.2.3 JSON

```
int const File_ID::JSON = 3 [static]
```

Definition at line [16](#) of file [File_ID.hpp](#).

The documentation for this class was generated from the following file:

- [File_ID.hpp](#)

4.6 FileBin Class Reference

Abstract interface for writing data to files.

```
#include <FileBin.hpp>
```

Public Member Functions

- virtual [~FileBin](#) ()=default
- virtual int [write](#) ()=0
- virtual void [setOutputFile](#) (std::string output)=0
- virtual void [setRawData](#) (std::string raw)=0
- virtual void [setFileName](#) (std::string fn)=0

4.6.1 Detailed Description

Abstract interface for writing data to files.

Definition at line 18 of file [FileBin.hpp](#).

4.6.2 Constructor & Destructor Documentation

4.6.2.1 ~FileBin()

```
virtual FileBin::~~FileBin () [virtual], [default]
```

Virtual destructor for proper cleanup in derived classes.

4.6.3 Member Function Documentation

4.6.3.1 setFileName()

```
virtual void FileBin::setFileName (
    std::string fn) [pure virtual]
```

Sets the output file name (without directory path).

Parameters

<i>fn</i>	File name for output
-----------	----------------------

4.6.3.2 setOutputFile()

```
virtual void FileBin::setOutputFile (
    std::string output) [pure virtual]
```

Sets the output file directory path for [write\(\)](#).

Parameters

<i>output</i>	Directory path where file will be written
---------------	---

4.6.3.3 setRawData()

```
virtual void FileBin::setRawData (
    std::string raw) [pure virtual]
```

Sets the raw data string to be written to file.

Parameters

<i>raw</i>	Formatted data string to write
------------	--------------------------------

4.6.3.4 write()

```
virtual int FileBin::write () [pure virtual]
```

Writes configured data to the specified output file.

Creates or overwrites the file at the configured output path and writes the configured raw data to it.

Returns

Status code (0 on success, non-zero on failure)

The documentation for this class was generated from the following file:

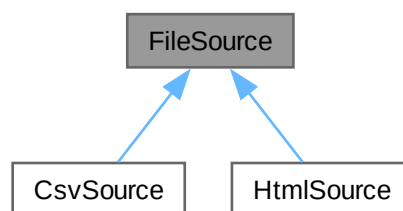
- [FileBin.hpp](#)

4.7 FileSource Class Reference

Abstract interface for reading file source data.

```
#include <FileSource.hpp>
```

Inheritance diagram for FileSource:



Public Member Functions

- virtual std::list< std::string > [read](#) ()=0
- virtual void [setSource](#) (std::string src)=0
- virtual int [getSourceId](#) ()=0

4.7.1 Detailed Description

Abstract interface for reading file source data.

Definition at line 18 of file [FileSource.hpp](#).

4.7.2 Member Function Documentation

4.7.2.1 getSourceId()

```
virtual int FileSource::getSourceId () [pure virtual]
```

Gets the file format ID associated with this source.

Returns

Integer ID representing the file format (e.g., `File_ID::CSV`)

Implemented in [CsvSource](#), and [HtmlSource](#).

4.7.2.2 read()

```
virtual std::list< std::string > FileSource::read () [pure virtual]
```

Reads the configured file source path and returns raw file data.

Returns

List of strings containing raw file data (line-by-line or complete content)

Implemented in [CsvSource](#), and [HtmlSource](#).

4.7.2.3 setSource()

```
virtual void FileSource::setSource (
    std::string src) [pure virtual]
```

Sets the file source path for [read\(\)](#) to use.

Parameters

<i>src</i>	File path to configure as the source for reading
------------	--

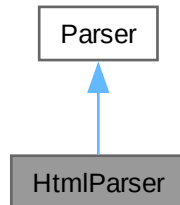
The documentation for this class was generated from the following file:

- [FileSource.hpp](#)

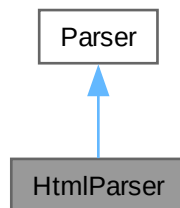
4.8 HtmlParser Class Reference

HTML file parser implementation using Gumbo.

Inheritance diagram for HtmlParser:



Collaboration diagram for HtmlParser:



Public Member Functions

- [HtmlParser](#) (list< string > raw_data)
- list< unordered_map< string, string > > [parse](#) ()
- void [setData](#) (list< string > data) override
- int [getParseId](#) () override

Public Member Functions inherited from [Parser](#)

- virtual void [setData](#) (std::list< std::string > data)=0

Private Member Functions

- void [extract_links](#) (GumboNode *node, list< unordered_map< string, string > > &out)
- string [get_text](#) (GumboNode *node)

Private Attributes

- list< string > [raw_data](#) = {}
- int const [parse_id](#) = File_ID::HTML

4.8.1 Detailed Description

HTML file parser implementation using Gumbo.

Definition at line 16 of file [HtmlParser.cpp](#).

4.8.2 Constructor & Destructor Documentation

4.8.2.1 HtmlParser() [1/2]

```
HtmlParser::HtmlParser (  
    list< string > raw_data) [inline]
```

Definition at line 18 of file [HtmlParser.cpp](#).

4.8.2.2 HtmlParser() [2/2]

```
HtmlParser::HtmlParser () [inline]
```

Definition at line 19 of file [HtmlParser.cpp](#).

4.8.3 Member Function Documentation

4.8.3.1 extract_links()

```
void HtmlParser::extract_links (  
    GumboNode * node,  
    list< unordered_map< string, string > > & out) [inline], [private]
```

Definition at line 56 of file [HtmlParser.cpp](#).

4.8.3.2 get_text()

```
string HtmlParser::get_text (  
    GumboNode * node) [inline], [private]
```

Definition at line 86 of file [HtmlParser.cpp](#).

4.8.3.3 getParsed()

```
int HtmlParser::getId () [inline], [override], [virtual]
```

Gets the file format ID associated with this parser.

Returns

Integer ID representing the file format (e.g., File_ID::CSV)

Implements [Parser](#).

Definition at line 48 of file [HtmlParser.cpp](#).

4.8.3.4 parse()

```
list< unordered_map< string, string > > HtmlParser::parse () [inline], [virtual]
```

Parses raw file data into a structured list of dictionaries.

Converts raw file data configured via setData() into a list of unordered string-to-string dictionaries. Each dictionary represents a row with column titles as keys and row contents as values.

Returns

List of unordered maps representing parsed file rows

Implements [Parser](#).

Definition at line 22 of file [HtmlParser.cpp](#).

4.8.3.5 setData()

```
void HtmlParser::setData (
    list< string > data) [inline], [override]
```

Definition at line 46 of file [HtmlParser.cpp](#).

4.8.4 Member Data Documentation

4.8.4.1 parse_id

```
int const HtmlParser::parse_id = File_ID::HTML [private]
```

Definition at line 53 of file [HtmlParser.cpp](#).

4.8.4.2 raw_data

```
list<string> HtmlParser::raw_data = {} [private]
```

Definition at line 52 of file [HtmlParser.cpp](#).

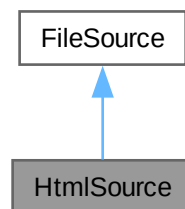
The documentation for this class was generated from the following file:

- [HtmlParser.cpp](#)

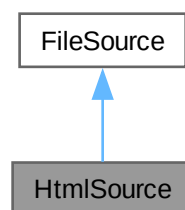
4.9 HtmlSource Class Reference

HTML file source/reader implementation.

Inheritance diagram for HtmlSource:



Collaboration diagram for HtmlSource:



Public Member Functions

- [HtmlSource](#) (std::string s)
- list< string > [read](#) () override
- int [getSourceId](#) ()
- void [setSource](#) (string new_source)

Public Member Functions inherited from [FileSource](#)

- virtual void [setSource](#) (std::string src)=0

Private Attributes

- string [src_path](#) = ""
- int const [source_id](#) = File_ID::HTML

4.9.1 Detailed Description

HTML file source/reader implementation.

Definition at line 16 of file [HtmlSource.cpp](#).

4.9.2 Constructor & Destructor Documentation

4.9.2.1 HtmlSource()

```
HtmlSource::HtmlSource (
    std::string s) [inline]
```

Definition at line 18 of file [HtmlSource.cpp](#).

4.9.3 Member Function Documentation

4.9.3.1 getSourceId()

```
int HtmlSource::getSourceId () [inline], [virtual]
```

Gets the file format ID associated with this source.

Returns

Integer ID representing the file format (e.g., File_ID::CSV)

Implements [FileSource](#).

Definition at line 42 of file [HtmlSource.cpp](#).

4.9.3.2 read()

```
list< string > HtmlSource::read () [inline], [override], [virtual]
```

Reads the configured file source path and returns raw file data.

Returns

List of strings containing raw file data (line-by-line or complete content)

Implements [FileSource](#).

Definition at line 21 of file [HtmlSource.cpp](#).

4.9.3.3 setSource()

```
void HtmlSource::setSource (
    string new_source) [inline]
```

Definition at line 43 of file [HtmlSource.cpp](#).

4.9.4 Member Data Documentation

4.9.4.1 source_id

```
int const HtmlSource::source_id = File_ID::HTML [private]
```

Definition at line 47 of file [HtmlSource.cpp](#).

4.9.4.2 src_path

```
string HtmlSource::src_path = " " [private]
```

Definition at line 46 of file [HtmlSource.cpp](#).

The documentation for this class was generated from the following file:

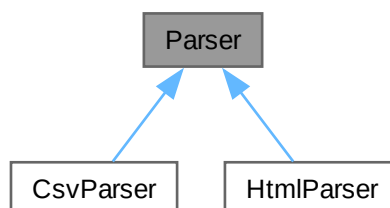
- [HtmlSource.cpp](#)

4.10 Parser Class Reference

Abstract interface for parsing file data into structured format.

```
#include <Parser.hpp>
```

Inheritance diagram for Parser:



Public Member Functions

- virtual `std::list< std::unordered_map< std::string, std::string > >` [parse](#) ()=0
- virtual void [setData](#) (`std::list< std::string >` data)=0
- virtual int [getParseId](#) ()=0

4.10.1 Detailed Description

Abstract interface for parsing file data into structured format.

Definition at line 20 of file [Parser.hpp](#).

4.10.2 Member Function Documentation

4.10.2.1 [getParseId\(\)](#)

```
virtual int Parser::getParseId () [pure virtual]
```

Gets the file format ID associated with this parser.

Returns

Integer ID representing the file format (e.g., `File_ID::CSV`)

Implemented in [CsvParser](#), and [HtmlParser](#).

4.10.2.2 [parse\(\)](#)

```
virtual std::list< std::unordered_map< std::string, std::string > > Parser::parse () [pure virtual]
```

Parses raw file data into a structured list of dictionaries.

Converts raw file data configured via [setData\(\)](#) into a list of unordered string-to-string dictionaries. Each dictionary represents a row with column titles as keys and row contents as values.

Returns

List of unordered maps representing parsed file rows

Implemented in [CsvParser](#), and [HtmlParser](#).

4.10.2.3 [setData\(\)](#)

```
virtual void Parser::setData (  
    std::list< std::string > data) [pure virtual]
```

Sets the raw file data to be parsed by [parse\(\)](#).

Parameters

<i>data</i>	List of strings containing raw file data
-------------	--

The documentation for this class was generated from the following file:

- [Parser.hpp](#)

4.11 WatchLaterParser Class Reference

Specialized parser for YouTube Takeout Watch Later CSV files.

```
#include <WatchLaterParser.hpp>
```

Public Member Functions

- [WatchLaterParser](#) (const std::string &filePath)
- std::list< Video > [parse](#) ()

Private Attributes

- std::string [filePath](#)

4.11.1 Detailed Description

Specialized parser for YouTube Takeout Watch Later CSV files.

Definition at line 22 of file [WatchLaterParser.hpp](#).

4.11.2 Constructor & Destructor Documentation

4.11.2.1 WatchLaterParser()

```
WatchLaterParser::WatchLaterParser (  
    const std::string & filePath) [inline], [explicit]
```

Constructs [WatchLaterParser](#) for the given CSV file.

Parameters

<i>filePath</i>	Path to the YouTube Takeout Watch Later CSV file
-----------------	--

Definition at line 27 of file [WatchLaterParser.hpp](#).

4.11.3 Member Function Documentation

4.11.3.1 parse()

```
std::list< Video > WatchLaterParser::parse () [inline]
```

Reads and parses the Watch Later CSV file into Video objects.

Each returned Video object has only the videoID field populated; all other fields (title, channelID, etc.) are empty/zero.

Returns

List of Video objects initialized from Watch Later CSV rows

Definition at line 36 of file [WatchLaterParser.hpp](#).

4.11.4 Member Data Documentation

4.11.4.1 filePath

```
std::string WatchLaterParser::filePath [private]
```

Definition at line 61 of file [WatchLaterParser.hpp](#).

The documentation for this class was generated from the following file:

- [WatchLaterParser.hpp](#)

Chapter 5

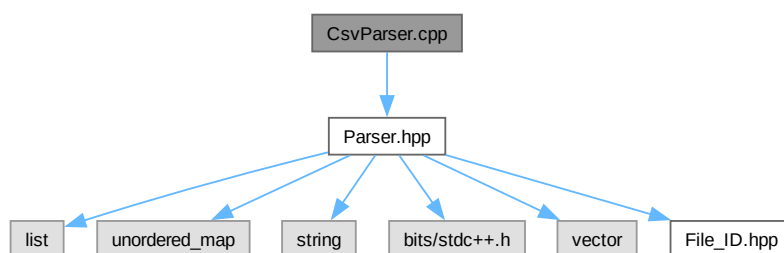
File Documentation

5.1 CsvParser.cpp File Reference

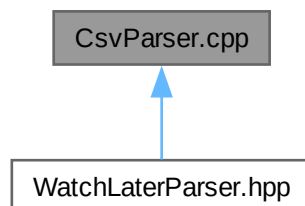
CSV file parser implementation.

```
#include "Parser.hpp"
```

Include dependency graph for CsvParser.cpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [CsvParser](#)
CSV file parser implementation.

5.1.1 Detailed Description

CSV file parser implementation.

Author

Shamar Pennant

Implements the [Parser](#) interface for CSV files. Parses comma-separated values into `list<Dict<str,str>>` format where the first row is treated as column headers.

Definition in file [CsvParser.cpp](#).

5.2 CsvParser.cpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #include "Parser.hpp"
00011
00012 using namespace std;
00013
00015 class CsvParser : public Parser {
00016     public:
00017         CsvParser() {}
00018         CsvParser(list<string> raw_data) {
00019             this->raw_data = raw_data;
00020         }
00021         ~CsvParser() override = default;
00022
00023     list<unordered_map<string, string>> parse() override {
00024         // check if raw_data is empty
00025         if (raw_data.empty()) {
00026             cerr << "error: raw data string is empty in CsvParser";
00027             exit(1);
00028         }
00029
00030         // returned value at end of function
00031         list<unordered_map<string, string>> ret;
00032
00033         // header contents
00034         vector<string> col_titles;
00035
00036         int line_num = 0;
00037         // loop through the raw_data list until it's empty
00038         while (!raw_data.empty()) {
00039             // grab the next line and advance
00040             string line = raw_data.front();
00041             raw_data.pop_front();
00042
00043             if (line_num == 0) {
00044                 /*
00045                  When on the first line (the col titles)
00046                  save their names within col_titles to be
00047                  used later. formatting each dictionary
00048                  contents as key = col_title and value =
00049                  row_entry
00050                  */
00051                 stringstream temp_ss(line);
00052                 string t = "";
00053                 while (getline(temp_ss, t, ',')) {
00054                     // trim any white space/newline char
00055                     if (!t.empty() && t.back() == '\n') {t.pop_back();}

```

```

00056             // save col
00057             col_titles.push_back(t);
00058         }
00059     } else { // for any other line, they're normal rows
00060         int num_col = 0;
00061         unordered_map<string, string> row_contents;
00062
00063         /* delimit the line/row and place key value pay
00064         <"col_name", "row_value"> inside the dictionary*/
00065         stringstream temp_ss(line);
00066         string t = "";
00067         while (getline(temp_ss, t, ',')) {
00068             if (!t.empty() && t.back() == '\\n') {t.pop_back();}
00069             row_contents[col_titles.at(num_col++)] = t;
00070         }
00071
00072         // add the dictionary as a line to ret
00073         ret.push_back(row_contents);
00074     }
00075
00076     line_num++;
00077 }
00078
00079 return ret;
00080 }
00081
00082 void setData(list<string> data) override {
00083     this->raw_data = data;
00084 }
00085 int getParseId() override {return parse_id;}
00086
00087 private:
00088     list<string> raw_data;
00089     int parse_id = File_ID::CSV;
00090 };

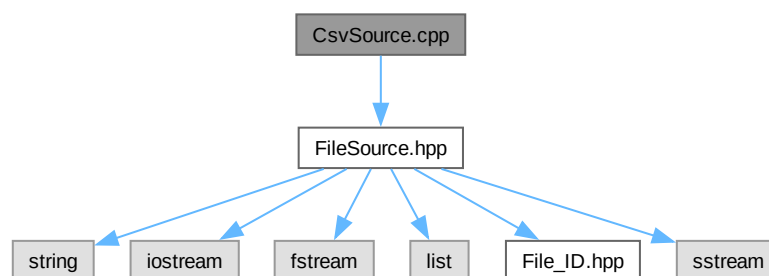
```

5.3 CsvSource.cpp File Reference

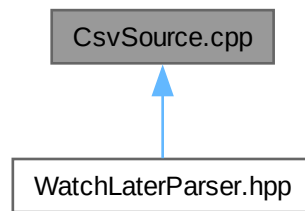
CSV file source/reader implementation.

```
#include "FileSource.hpp"
```

Include dependency graph for CsvSource.cpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [CsvSource](#)
CSV file source/reader implementation.

5.3.1 Detailed Description

CSV file source/reader implementation.

Author

Shamar Pennant

Implements the [FileSource](#) interface for reading CSV files. Reads file contents line-by-line and returns as `list<string>`.

Definition in file [CsvSource.cpp](#).

5.4 CsvSource.cpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #include "FileSource.hpp"
00011
00012 using namespace std;
00013
00015 class CsvSource : public FileSource {
00016     public:
00017         CsvSource(string src) {
00018
00019             this-> src = src;
00020         }
00021         ~CsvSource() override = default;
00022
00023         list<string> read() override {
00024             if (src.empty()) { // in the event that src is empty
00025                 cerr << "error: src string is empty in CsvSource";
00026                 exit(1);
00027             }
00028

```

```

00029         // returned string container
00030         list<string> ret;
00031
00032         // creating (in) file stream with given file source path
00033         fstream read_file_in(src, ios::in);
00034         if (!read_file_in.is_open()) {
00035             cerr << "unable to open " << src << " within CsvSource" << endl;
00036             exit(1);
00037         }
00038
00039         if (!read_file_in.is_open()) {
00040             cerr << "error: could not open file: " << src;
00041             exit(1);
00042         }
00043
00044         // holds file line
00045         string line = "";
00046
00047         // reading the opened file stream into a temporary string
00048         while (getline(read_file_in, line)) {ret.push_back(line);}
00049
00050         return ret;
00051     }
00052
00053     void setSource(string src) override {this->src = src;}
00054     int getSourceId() override {return source_id;}
00055
00056     private:
00057         string src = ""; // holds the file path
00058         int source_id = File_ID::CSV; // holds this objects id to match with a Parser object
00059 };

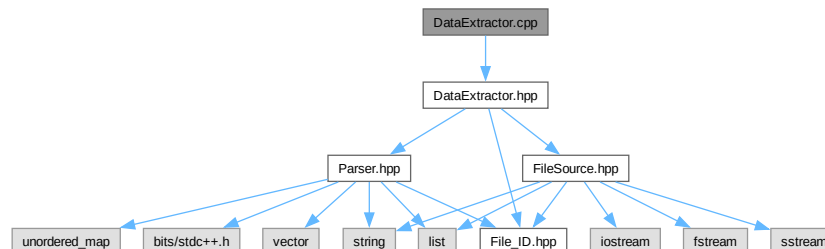
```

5.5 DataExtractor.cpp File Reference

[DataExtractor](#) class implementation.

```
#include "DataExtractor.hpp"
```

Include dependency graph for DataExtractor.cpp:



5.5.1 Detailed Description

[DataExtractor](#) class implementation.

Author

Shamar Pennant

Implements the [DataExtractor](#) class which coordinates [FileSource](#) and [Parser](#) objects to read and parse file data into structured format.

Definition in file [DataExtractor.cpp](#).

5.6 DataExtractor.cpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #include "DataExtractor.hpp"
00011
00012 using namespace std;
00013
00014 DataExtractor::DataExtractor(unique_ptr<FileSource> src, unique_ptr<Parser> p) {
00015     // change ownership of the param to this objects instance
00016     src_format = move(src);
00017     par_format = move(p);
00018 }
00019
00020 DataExtractor::~DataExtractor() {}
00021
00022 list<unordered_map<string, string> > DataExtractor::extract() {
00023     // grab the respective file format ids
00024     int source_id_in_use = src_format->getSourceId();
00025     int parse_id_in_use = par_format->getParseId();
00026
00027     // check if either id's are out of bounds (they shouldn't be)
00028     if (source_id_in_use <= -1 || parse_id_in_use <= -1) {
00029         cerr << "One of the provided ID's is out of bounds:\nsrc_id = " << source_id_in_use <<
00030             "\nparse_id = " << parse_id_in_use;
00031         exit(1);
00032     }
00033
00034     // check if id's match (how we know they match formats)
00035     if (source_id_in_use != parse_id_in_use) {
00036         cerr << "selected FileSource and Parser id's don't match: \nsrc_id = " << source_id_in_use <<
00037             "\nparse_id = " << parse_id_in_use;
00038         exit(1);
00039     }
00040
00041     // ensure neither pointers are null
00042     if (src_format == nullptr || par_format == nullptr) {
00043         cerr << "Unable to find FileSource or Parser in extract():\nsrc_id = "
00044             << source_id_in_use << ", " << (src_format == nullptr ? "contains nothing (nullptr)" : "has
00045             obj");
00046         << "\nparse_id = "
00047             << parse_id_in_use << ", " << (par_format == nullptr ? "contains nothing (nullptr)" : "has
00048             obj");
00049         exit(1);
00050     }
00051
00052     // --- assuming the both source and parser are already configured ---
00053
00054     // grab the raw data from the file
00055     auto raw_data = src_format->read();
00056     // feed the raw data into the given parser
00057     par_format->setData(raw_data);
00058     // format the data with the respective parser
00059     list<unordered_map<string, string> > formatted_data = par_format->parse();
00060
00061     return formatted_data;
00062 }
00063
00064 void DataExtractor::setParser(unique_ptr<Parser> parser) {
00065     par_format = move(parser);
00066 }
00067
00068 void DataExtractor::setSource(unique_ptr<FileSource> src) {
00069     src_format = move(src);
00070 }
00071

```

5.7 DataExtractor.hpp File Reference

Extracts raw file data and parses it into structured format.

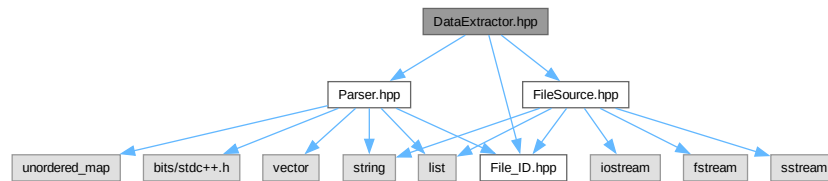
```

#include "FileSource.hpp"
#include "Parser.hpp"

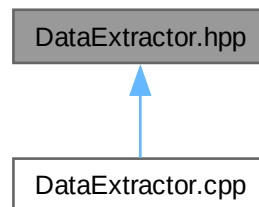
```

```
#include "File_ID.hpp"
```

Include dependency graph for DataExtractor.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [DataExtractor](#)
Coordinates [FileSource](#) and [Parser](#) to extract and parse file data.

5.7.1 Detailed Description

Extracts raw file data and parses it into structured format.

Author

Shamar Pennant

Coordinates file reading (via [FileSource](#)) and data parsing (via [Parser](#)) to produce a consistent `list<Dict<str,str>>` format. Validates that [FileSource](#) and [Parser](#) objects have matching file format IDs before processing.

Example usage:

```

auto source = std::make_unique<CsvSource>("data.csv");
auto parser = std::make_unique<CsvParser>();
DataExtractor extractor(std::move(source), std::move(parser));
auto data = extractor.extract();
  
```

Definition in file [DataExtractor.hpp](#).

5.8 DataExtractor.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include "FileSource.hpp"
00004 #include "Parser.hpp"
00005 #include "File_ID.hpp"
00006
00025 class DataExtractor {
00026     public:
00032         DataExtractor(std::unique_ptr<FileSource> src, std::unique_ptr<Parser> p);
00033
00035         ~DataExtractor();
00036
00046         std::list<std::unordered_map<std::string, std::string>> extract();
00047
00051         void setSource(std::unique_ptr<FileSource> src);
00052
00056         void setParser(std::unique_ptr<Parser> p);
00057
00058     private:
00059         std::unique_ptr<FileSource> src_format;
00060         std::unique_ptr<Parser> par_format;
00061 };

```

5.9 DataSaver.hpp File Reference

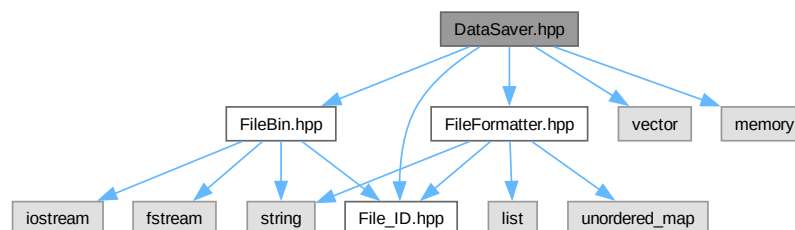
Saves structured data to files in desired format.

```

#include "FileBin.hpp"
#include "FileFormatter.hpp"
#include <vector>
#include <memory>
#include "File_ID.hpp"

```

Include dependency graph for DataSaver.hpp:



Classes

- class [DataSaver](#)

Coordinates [FileFormatter](#) and [FileBin](#) to save structured data to files.

5.9.1 Detailed Description

Saves structured data to files in desired format.

Author

Shamar Pennant

Coordinates data formatting (via FileFormatter) and file output (via FileBin) to persist list<Dict<str,str>> data to files. Maintains collections of FileBin and FileFormatter objects to support multiple file formats.

Definition in file DataSaver.hpp.

5.10 DataSaver.hpp

[Go to the documentation of this file.](#)

```

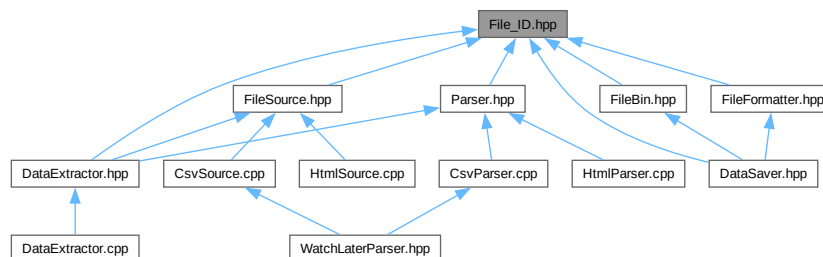
00001 #pragma once
00002
00003 #include "FileBin.hpp"
00004 #include "FileFormatter.hpp"
00005 #include <vector>
00006 #include <memory>
00007 #include "File_ID.hpp"
00008
00019 class DataSaver {
00020     public:
00025         DataSaver(FileBin b, FileFormatter ff);
00026
00028         ~DataSaver();
00029
00034         int save(std::list<std::unordered_map<std::string, std::string>> data);
00035
00039         void setFileBin(FileBin b);
00040
00044         void setFileFormatter(FileFormatter ff);
00045
00046     private:
00047         // contains vector of all used FileBin objects
00048         std::vector<std::unique_ptr<FileBin>> bins;
00049         // contains vectors of all used FileFormatters
00050         std::vector<std::unique_ptr<FileFormatter>> formatters;
00051 };

```

5.11 File_ID.hpp File Reference

File format ID constants.

This graph shows which files directly or indirectly include this file:



Classes

- class [File_ID](#)

File format ID constants for validating source-parser compatibility.

5.11.1 Detailed Description

File format ID constants.

Author

Shamar Pennant

Defines static integer constants mapping file types to unique identifiers. Used to validate that [FileSource](#) and [Parser](#) objects are compatible.

Definition in file [File_ID.hpp](#).

5.12 File_ID.hpp

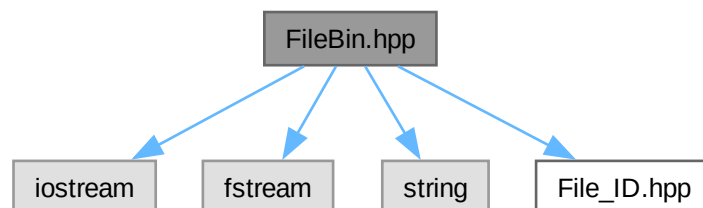
[Go to the documentation of this file.](#)

```
00001
00009 #pragma once
00010
00012 class File_ID {
00013     public:
00014         static int const CSV = 1;
00015         static int const HTML = 2;
00016         static int const JSON = 3;
00017 };
```

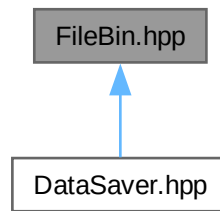
5.13 FileBin.hpp File Reference

Abstract base class for writing data to files.

```
#include <iostream>
#include <fstream>
#include <string>
#include "File_ID.hpp"
Include dependency graph for FileBin.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [FileBin](#)
Abstract interface for writing data to files.

5.13.1 Detailed Description

Abstract base class for writing data to files.

Author

Shamar Pennant

Defines the interface for persisting formatted string data to files. Subclasses implement format-specific file writing operations (e.g., CSV files, HTML files, etc.).

Definition in file [FileBin.hpp](#).

5.14 FileBin.hpp

[Go to the documentation of this file.](#)

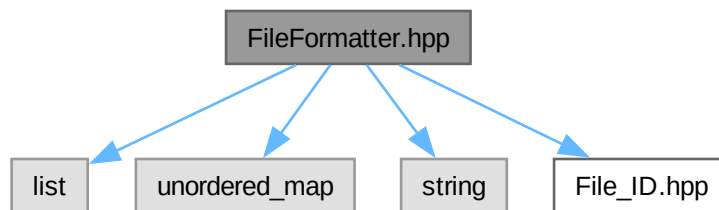
```

00001 #pragma once
00002
00003 #include <iostream>
00004 #include <fstream>
00005 #include <string>
00006 #include "File_ID.hpp"
00007
00018 class FileBin {
00019     public:
00021         virtual ~FileBin() = default;
00022
00030         virtual int write() = 0;
00031
00035         virtual void setOutputFile(std::string output) = 0;
00036
00040         virtual void setRawData(std::string raw) = 0;
00041
00045         virtual void setFileName(std::string fn) = 0;
00046
00047     protected:
00048         FileBin() = default;
00049 };
  
```

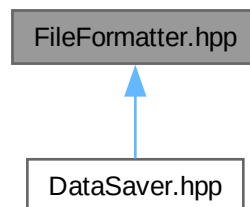
5.15 FileFormatter.hpp File Reference

Abstract base class for formatting structured data to file format.

```
#include <list>
#include <unordered_map>
#include <string>
#include "File_ID.hpp"
Include dependency graph for FileFormatter.hpp:
```



This graph shows which files directly or indirectly include this file:



Functions

- virtual `~FileFormatter()`=default
Abstract interface for formatting structured data to file format.class FileFormatter {.
- virtual `std::string format()`=0
- virtual `void setData(std::list< std::unordered_map< std::string, std::string > > data)`=0
- **FileFormatter**`()`=default

5.15.1 Detailed Description

Abstract base class for formatting structured data to file format.

Author

Shamar Pennant

Converts `list<Dict<str,str>>` structured data into format-specific string representation (e.g., CSV, HTML, JSON). Subclasses implement format-specific formatting logic for different file types.

Definition in file [FileFormatter.hpp](#).

5.15.2 Function Documentation

5.15.2.1 `format()`

```
virtual std::string format () [pure virtual]
```

Formats structured data into format-specific string representation.

Converts `list<Dict<str,str>>` configured via [setData\(\)](#) into a properly formatted string for writing to file (e.g., CSV with commas and newlines).

Returns

Formatted string ready for file output

5.15.2.2 `setData()`

```
virtual void setData (
    std::list< std::unordered_map< std::string, std::string > > data) [pure virtual]
```

Sets the structured data to be formatted by [format\(\)](#).

Parameters

<i>data</i>	List of dictionaries representing data rows
-------------	---

5.15.2.3 `~FileFormatter()`

```
virtual ~FileFormatter () [virtual], [default]
```

Abstract interface for formatting structured data to file format.`class FileFormatter {.`

Virtual destructor for proper cleanup in derived classes.

5.16 FileFormatter.hpp

[Go to the documentation of this file.](#)

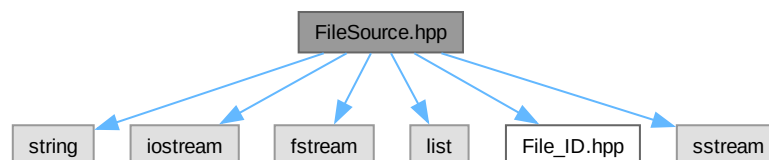
```
00001 #pragma once
00002
00003 #include <list>
00004 #include <unordered_map>
00005 #include <string>
00006 #include "File_ID.hpp"
00007
00017     public:
00019         virtual ~FileFormatter() = default;
00020
00028         virtual std::string format() = 0;
00029
00033         virtual void setData(std::list<std::unordered_map<std::string, std::string>> data) = 0;
00034
00035     protected:
00036         FileFormatter() = default;
00037 };
```

5.17 FileSource.hpp File Reference

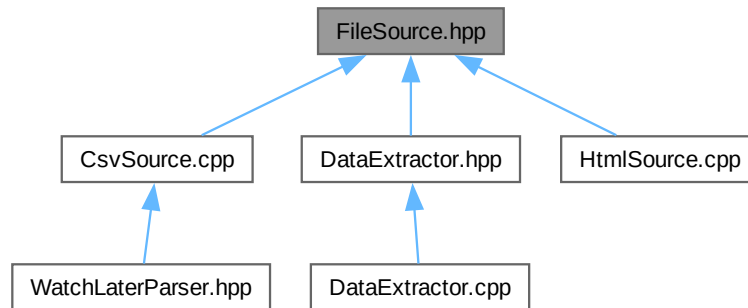
Abstract base class for reading file source data.

```
#include <string>
#include <iostream>
#include <fstream>
#include <list>
#include "File_ID.hpp"
#include <sstream>
```

Include dependency graph for FileSource.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [FileSource](#)
Abstract interface for reading file source data.

5.17.1 Detailed Description

Abstract base class for reading file source data.

Author

Shamar Pennant

Defines the interface for file reading operations across different file formats. Subclasses implement format-specific source reading (CSV, HTML, etc.).

Definition in file [FileSource.hpp](#).

5.18 FileSource.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include <string>
00003 #include <iostream>
00004 #include <fstream>
00005 #include <list>
00006 #include "File_ID.hpp"
00007 #include <sstream>
00008
00018 class FileSource {
00019     public:
00020         virtual ~FileSource() = default;
00021
00025         virtual std::list<std::string> read() = 0;
00026
00030         virtual void setSource(std::string src) = 0;
00031
00035         virtual int getSourceId() = 0;
00036
00037     protected:
00038         FileSource() = default;
00039 };
  
```

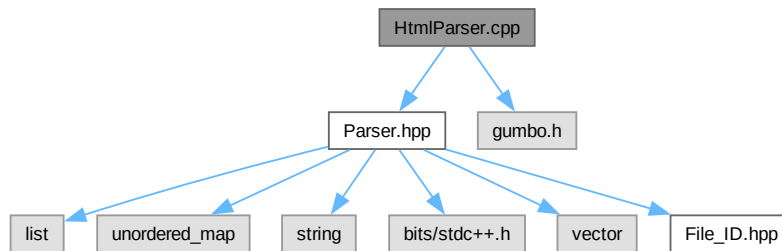
5.19 HtmlParser.cpp File Reference

HTML file parser implementation using Gumbo.

```
#include "Parser.hpp"
```

```
#include <gumbo.h>
```

Include dependency graph for HtmlParser.cpp:



Classes

- class [HtmlParser](#)

HTML file parser implementation using Gumbo.

5.19.1 Detailed Description

HTML file parser implementation using Gumbo.

Author

Shamar Pennant

Implements the [Parser](#) interface for HTML files using the Gumbo HTML5 parser. Extracts HTML links and their text content into list<Dict<str,str>> format.

Definition in file [HtmlParser.cpp](#).

5.20 HtmlParser.cpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #include "Parser.hpp"
00011 #include <gumbo.h>
00012
00013 using namespace std;
00014
00016 class HtmlParser : public Parser {
00017     public:
00018         HtmlParser(list<string> raw_data) {this->raw_data = raw_data;}
00019         HtmlParser() {}
  
```

```

00020     ~HtmlParser() = default;
00021
00022     list<unordered_map<string,string> parse() {
00023         if (raw_data.empty()) {
00024             cerr << "HtmlParser: raw_data is empty" << endl;
00025             exit(1);
00026         }
00027
00028         // given working with gumbo, everything is within a single item in the provided list
00029         string html = raw_data.front();
00030
00031         //grab the output through thr gumbo parser
00032         GumboOutput* output = gumbo_parse(html.c_str());
00033
00034         // container for returned results
00035         list<unordered_map<string,string> results;
00036
00037         // place extracted links from the output into results
00038         extract_links(output->root, results);
00039
00040         // destroy ptr
00041         gumbo_destroy_output(&kGumboDefaultOptions, output);
00042
00043         return results;
00044     }
00045
00046     void setData(list<string> data) override {raw_data = data;}
00047
00048     int getParseId() override {return parse_id;}
00049
00050 private:
00051     list<string> raw_data = {};
00052     int const parse_id = File_ID::HTML;
00053
00054     // helper funtion to extracts links and tags using gumbo
00055     void extract_links(GumboNode* node, list<unordered_map<string,string>& out) {
00056         if (node->type != GUMBO_NODE_ELEMENT)
00057             return;
00058
00059         GumboElement& el = node->v.element;
00060
00061         if (el.tag == GUMBO_TAG_A) {
00062             unordered_map<string,string> row;
00063
00064             // Extract href attribute
00065             GumboAttribute* href = gumbo_get_attribute(&el.attributes, "href");
00066             if (href) row["href"] = href->value;
00067
00068             // Extract visible text
00069             row["text"] = get_text(node);
00070
00071             // Tag name for debugging/consistency
00072             row["tag"] = "a";
00073
00074             out.push_back(row);
00075         }
00076
00077         // Recurse into children
00078         GumboVector* children = &el.children;
00079         for (unsigned i = 0; i < children->length; i++) {
00080             extract_links(static_cast<GumboNode*>(children->data[i]), out);
00081         }
00082     }
00083
00084     // Extract all text under a node
00085     string get_text(GumboNode* node) {
00086         if (node->type == GUMBO_NODE_TEXT)
00087             return node->v.text.text;
00088
00089         if (node->type != GUMBO_NODE_ELEMENT)
00090             return "";
00091
00092         string result;
00093         GumboVector* children = &node->v.element.children;
00094         for (unsigned i = 0; i < children->length; i++) {
00095             result += get_text(static_cast<GumboNode*>(children->data[i]));
00096         }
00097         return result;
00098     }
00099 }
00100 };

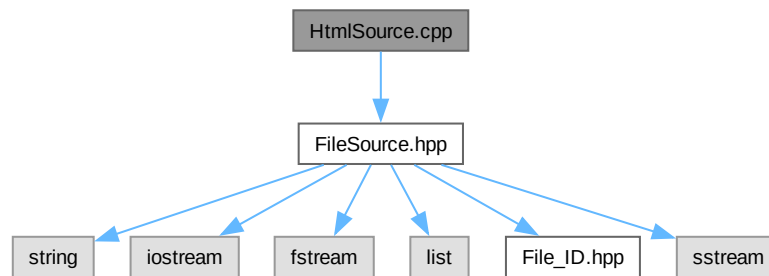
```

5.21 HtmlSource.cpp File Reference

HTML file source/reader implementation.

```
#include "FileSource.hpp"
```

Include dependency graph for HtmlSource.cpp:



Classes

- class [HtmlSource](#)
HTML file source/reader implementation.

5.21.1 Detailed Description

HTML file source/reader implementation.

Author

Shamar Pennant

Implements the [FileSource](#) interface for reading HTML files. Reads entire HTML file contents as a single string in `list<string>`. Note: Returns list with single item due to Gumbo parser requirements.

Definition in file [HtmlSource.cpp](#).

5.22 HtmlSource.cpp

[Go to the documentation of this file.](#)

```

00001
00010
00011 #include "FileSource.hpp"
00012
00013 using namespace std;
00014
00016 class HtmlSource : public FileSource {
00017     public:
00018         HtmlSource(std::string s) {src_path = s;}
00019         ~HtmlSource() = default;
00020

```

```

00021     list<string> read() override {
00022         // check if src is empty
00023         if (src_path.empty()) {
00024             cerr << "html source path is empty" << endl;
00025             exit(1);
00026         }
00027
00028         // open a file stream
00029         ifstream read_file_in(src_path, ios::in);
00030         if (!read_file_in.is_open()) {
00031             cerr << "unable to open " << src_path << " within HtmlSource" << endl;
00032             exit(1);
00033         }
00034
00035         // retrieve file contents
00036         stringstream buff;
00037         buff << read_file_in.rdbuf();
00038
00039         return {buff.str()};
00040     }
00041
00042     int getSourceId() {return source_id;}
00043     void setSource(string new_source) {src_path = new_source;}
00044
00045 private:
00046     string src_path = " ";
00047     int const source_id = File_ID::HTML;
00048 };

```

5.23 Parser.hpp File Reference

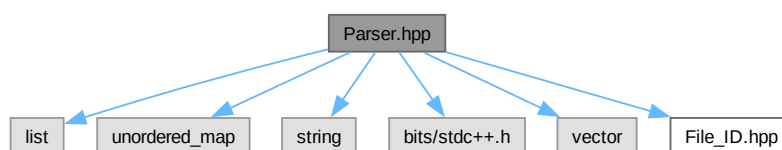
Abstract base class for parsing file data into structured format.

```

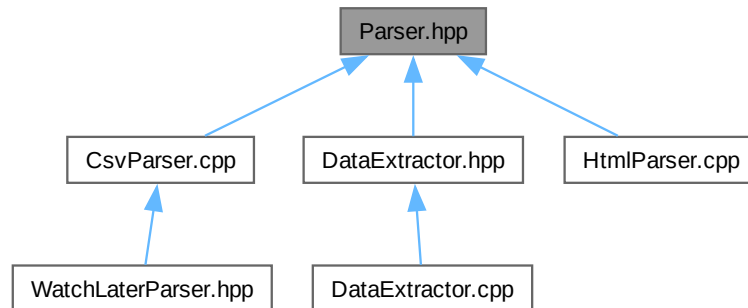
#include <list>
#include <unordered_map>
#include <string>
#include <bits/stdc++.h>
#include <vector>
#include "File_ID.hpp"

```

Include dependency graph for Parser.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [Parser](#)
Abstract interface for parsing file data into structured format.

5.23.1 Detailed Description

Abstract base class for parsing file data into structured format.

Author

Shamar Pennant

Defines the interface for converting raw file data into a consistent list of dictionaries format (list<↵ Dict<string,string>>). Each dictionary represents a row/entry with column headers as keys.

Definition in file [Parser.hpp](#).

5.24 Parser.hpp

[Go to the documentation of this file.](#)

```

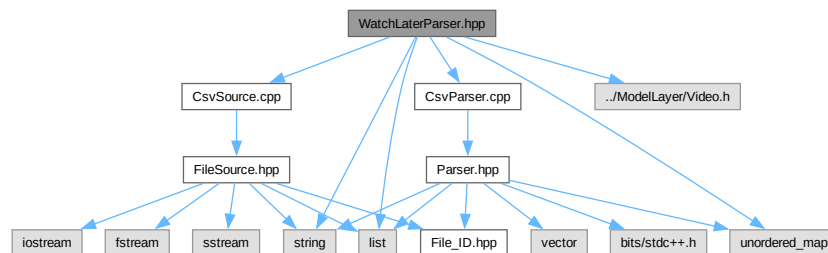
00001 #pragma once
00002
00003 #include <list>
00004 #include <unordered_map>
00005 #include <string>
00006 #include <bits/stdc++.h>
00007 #include <vector>
00008 #include "File_ID.hpp"
00009
00020 class Parser {
00021     public:
00022         virtual ~Parser() = default;
00023
00033         virtual std::list<std::unordered_map<std::string, std::string>> parse() = 0;
00034
00038         virtual void setData(std::list<std::string> data) = 0;
00039
00043         virtual int getParseId() = 0;
00044
00045     protected:
00046         Parser() = default;
00047 };
  
```

5.25 WatchLaterParser.hpp File Reference

Specialized parser for YouTube Takeout Watch Later CSV files.

```
#include "CsvSource.cpp"
#include "CsvParser.cpp"
#include "../ModelLayer/Video.h"
#include <list>
#include <string>
#include <unordered_map>
```

Include dependency graph for WatchLaterParser.hpp:



Classes

- class [WatchLaterParser](#)
Specialized parser for YouTube Takeout Watch Later CSV files.

5.25.1 Detailed Description

Specialized parser for YouTube Takeout Watch Later CSV files.

Author

Shamar Pennant

Parses YouTube Takeout 'Watch later-videos.csv' files into Video objects with only the videoID field populated. Internally uses [CsvSource](#) for reading and [CsvParser](#) for parsing. The resulting Videos have empty strings/zero for all other fields since Watch Later CSV only provides ID and timestamp.

Definition in file [WatchLaterParser.hpp](#).

5.26 WatchLaterParser.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include "CsvSource.cpp"
00004 #include "CsvParser.cpp"
00005 #include "../ModelLayer/Video.h"
00006
00007 #include <list>
00008 #include <string>
00009 #include <unordered_map>
00010
00022 class WatchLaterParser {
00023 public:
00027     explicit WatchLaterParser(const std::string& filePath) : filePath(filePath) {}
00028
00036     std::list<Video> parse() {
00037         CsvSource source(filePath);
00038         CsvParser parser(source.read());
00039
00040         std::list<std::unordered_map<std::string, std::string>> rows = parser.parse();
00041         std::list<Video> videos;
00042
00043         for (const auto& row : rows) {
00044             auto it = row.find("Video ID");
00045             if (it != row.end() && !it->second.empty()) {
00046                 videos.emplace_back(
00047                     it->second,           // videoID
00048                     "",                  // title (unknown at this stage)
00049                     "",                  // channelID
00050                     "",                  // thumbnailURL
00051                     0,                  // duration
00052                     std::list<std::string>{} // tags
00053                 );
00054             }
00055         }
00056
00057         return videos;
00058     }
00059
00060 private:
00061     std::string filePath;
00062 };

```

Index

- ~DataExtractor
 - DataExtractor, [13](#)
- ~DataSaver
 - DataSaver, [15](#)
- ~FileBin
 - FileBin, [18](#)
- ~FileFormatter
 - FileFormatter.hpp, [43](#)
- bins
 - DataSaver, [16](#)
- CSV
 - File_ID, [17](#)
- CsvParser, [7](#)
 - CsvParser, [8](#)
 - getParseld, [8](#)
 - parse, [8](#)
 - parse_id, [9](#)
 - raw_data, [9](#)
 - setData, [9](#)
- CsvParser.cpp, [31](#), [32](#)
- CsvSource, [10](#)
 - CsvSource, [11](#)
 - getSourceId, [11](#)
 - read, [11](#)
 - setSource, [11](#)
 - source_id, [12](#)
 - src, [12](#)
- CsvSource.cpp, [33](#), [34](#)
- DataExtractor, [12](#)
 - ~DataExtractor, [13](#)
 - DataExtractor, [12](#)
 - extract, [13](#)
 - par_format, [14](#)
 - setParser, [13](#)
 - setSource, [14](#)
 - src_format, [14](#)
- DataExtractor.cpp, [35](#), [36](#)
- DataExtractor.hpp, [36](#), [38](#)
- DataSaver, [14](#)
 - ~DataSaver, [15](#)
 - bins, [16](#)
 - DataSaver, [15](#)
 - formatters, [16](#)
 - save, [15](#)
 - setFileBin, [16](#)
 - setFileFormatter, [16](#)
- DataSaver.hpp, [38](#), [39](#)
- extract
 - DataExtractor, [13](#)
- extract_links
 - HtmlParser, [22](#)
- File_ID, [17](#)
 - CSV, [17](#)
 - HTML, [17](#)
 - JSON, [17](#)
- File_ID.hpp, [39](#), [40](#)
- FileBin, [17](#)
 - ~FileBin, [18](#)
 - setFileName, [18](#)
 - setOutputFile, [18](#)
 - setRawData, [18](#)
 - write, [19](#)
- FileBin.hpp, [40](#), [41](#)
- FileFormatter.hpp, [42](#), [44](#)
 - ~FileFormatter, [43](#)
 - format, [43](#)
 - setData, [43](#)
- filePath
 - WatchLaterParser, [29](#)
- FileSource, [19](#)
 - getSourceId, [20](#)
 - read, [20](#)
 - setSource, [20](#)
- FileSource.hpp, [44](#), [45](#)
- format
 - FileFormatter.hpp, [43](#)
- formatters
 - DataSaver, [16](#)
- get_text
 - HtmlParser, [22](#)
- getParseld
 - CsvParser, [8](#)
 - HtmlParser, [22](#)
 - Parser, [27](#)
- getSourceId
 - CsvSource, [11](#)
 - FileSource, [20](#)
 - HtmlSource, [25](#)
- HTML
 - File_ID, [17](#)
- HtmlParser, [21](#)
 - extract_links, [22](#)
 - get_text, [22](#)
 - getParseld, [22](#)

- HtmlParser, 22
- parse, 23
- parse_id, 23
- raw_data, 23
- setData, 23
- HtmlParser.cpp, 46
- HtmlSource, 24
 - getSourceId, 25
 - HtmlSource, 25
 - read, 25
 - setSource, 25
 - source_id, 26
 - src_path, 26
- HtmlSource.cpp, 48
- JSON
 - File_ID, 17
- par_format
 - DataExtractor, 14
- parse
 - CsvParser, 8
 - HtmlParser, 23
 - Parser, 27
 - WatchLaterParser, 29
- parse_id
 - CsvParser, 9
 - HtmlParser, 23
- Parser, 26
 - getParseld, 27
 - parse, 27
 - setData, 27
- Parser.hpp, 49, 50
- raw_data
 - CsvParser, 9
 - HtmlParser, 23
- read
 - CsvSource, 11
 - FileSource, 20
 - HtmlSource, 25
- save
 - DataSaver, 15
- setData
 - CsvParser, 9
 - FileFormatter.hpp, 43
 - HtmlParser, 23
 - Parser, 27
- setFileBin
 - DataSaver, 16
- setFileFormatter
 - DataSaver, 16
- setFileName
 - FileBin, 18
- setOutputFile
 - FileBin, 18
- setParser
 - DataExtractor, 13
- setRawData
 - FileBin, 18
- setSource
 - CsvSource, 11
 - DataExtractor, 14
 - FileSource, 20
 - HtmlSource, 25
- source_id
 - CsvSource, 12
 - HtmlSource, 26
- src
 - CsvSource, 12
- src_format
 - DataExtractor, 14
- src_path
 - HtmlSource, 26
- WatchLaterParser, 28
 - filePath, 29
 - parse, 29
 - WatchLaterParser, 28
- WatchLaterParser.hpp, 51, 52
- write
 - FileBin, 19